


```
!pip install ultralytics -q
import ultralytics
ultralytics.checks()  # confirms GPU is available
```

Ultralytics 8.4.69 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Setup complete ✅ (2 CPUs, 12.7 GB RAM, 47.1/112.6 GB disk)

```
from roboflow import Roboflow
import yaml
import os

rf = Roboflow(api_key="bZewy443VlofM60vQSMi")
project = rf.workspace("augmented-startups").project("vehicle-registration-plates-train")
version = project.version(1)
dataset = version.download("yolov8")

data_yaml_path = os.path.join(dataset.location, "data.yaml")
print("Dataset path:", dataset.location)

with open(data_yaml_path, "r") as f:
    data = yaml.safe_load(f)

print("nc:", data["nc"])
print("names:", data["names"])
```

```
loading Roboflow workspace...
loading Roboflow project...
Dataset path: /content/Vehicle-Registration-Plates-1
nc: 1
names: ['License_Plate']
```

```
from pathlib import Path
import yaml

root = Path(dataset.location)

# rewrite all label files so every object becomes class 0 ("plate")
for split in ["train", "valid", "test"]:
    label_dir = root / split / "labels"
    if not label_dir.exists():
        continue

    for label_file in label_dir.glob("*.txt"):
        new_lines = []
        for line in label_file.read_text().strip().splitlines():
            parts = line.split()
            if len(parts) < 5:
                continue
            parts[0] = "0"  # class id -> plate
            new_lines.append(" ".join(parts))
        label_file.write_text("\n".join(new_lines) + ("\n" if new_lines else ""))
```

```
# fix data.yaml
data_yaml_path = root / "data.yaml"
with open(data_yaml_path, "r") as f:
    data = yaml.safe_load(f)

data["nc"] = 1
data["names"] = ["plate"]

with open(data_yaml_path, "w") as f:
    yaml.safe_dump(data, f, sort_keys=False)

print("Converted to single-class plate dataset.")
```

Converted to single-class plate dataset.

```
print("Dataset path:", dataset.location)
print(dataset)
```

Dataset path: /content/Vehicle-Registration-Plates-1
<roboflow.core.dataset.Dataset object at 0x7efe5a78bef0>

```
from ultralytics import YOLO

model = YOLO("yolov8s.pt")

results = model.train(
    data=str(data_yaml_path),
    epochs=80,
    imgsz=640,
    batch=16,
    patience=20,
    device=0,
    project="/content/runs",
    name="roadeye_plate_single_class",
    flipplr=0.0,
    flipud=0.0,
    degrees=3.0,
    translate=0.08,
    scale=0.3,
    hsv_v=0.5,
    hsv_s=0.6,
    mosaic=0.5,
    erasing=0.2,
    optimizer="AdamW",
    lr0=0.001,
    cos_lr=True,
    plots=True,
)

print("Training complete!")
print(f"Best mAP50: {results.results_dict.get('metrics/mAP50(B)')}")
```

Ultralytics 8.4.68 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=False
Overriding model.yaml nc=80 with nc=1

	from	n	params	module
0	-1	1	928	ultralytics.nn.modules.conv.Conv
1	-1	1	18560	ultralytics.nn.modules.conv.Conv
2	-1	1	29056	ultralytics.nn.modules.block.C2f
3	-1	1	73984	ultralytics.nn.modules.conv.Conv
4	-1	2	197632	ultralytics.nn.modules.block.C2f
5	-1	1	295424	ultralytics.nn.modules.conv.Conv
6	-1	2	788480	ultralytics.nn.modules.block.C2f
7	-1	1	1180672	ultralytics.nn.modules.conv.Conv
8	-1	1	1838080	ultralytics.nn.modules.block.C2f
9	-1	1	656896	ultralytics.nn.modules.block.SPPF
10	-1	1	0	torch.nn.modules.upsampling.Upsample
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat
12	-1	1	591360	ultralytics.nn.modules.block.C2f
13	-1	1	0	torch.nn.modules.upsampling.Upsample
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat
15	-1	1	148224	ultralytics.nn.modules.block.C2f
16	-1	1	147712	ultralytics.nn.modules.conv.Conv
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat
18	-1	1	493056	ultralytics.nn.modules.block.C2f
19	-1	1	590336	ultralytics.nn.modules.conv.Conv
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat
21	-1	1	1969152	ultralytics.nn.modules.block.C2f
22	[15, 18, 21]	1	2116435	ultralytics.nn.modules.head.Detect

Model summary: 130 layers, 11,135,987 parameters, 11,135,971 gradients, 28.6 GFLOPs

Transferred 349/355 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

AMP: checks passed 

train: Fast image access  (ping: 0.0±0.0 ms, read: 15.2±5.1 MB/s, size: 27.2 KB)

train: Scanning /content/Vehicle-Registration-Plates-1/train/labels... 6176 images,

train: New cache created: /content/Vehicle-Registration-Plates-1/train/labels.cache

alumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3,

val: Fast image access  (ping: 0.0±0.0 ms, read: 10.4±7.2 MB/s, size: 16.1 KB)

val: Scanning /content/Vehicle-Registration-Plates-1/valid/labels... 1765 images, 0

val: New cache created: /content/Vehicle-Registration-Plates-1/valid/labels.cache

optimizer: AdamW(lr=0.001, momentum=0.937) with parameter groups 57 weight(decay=0.

Plotting labels to /content/runs/roadeye_plate_single_class/labels.jpg...

Image sizes 640 train, 640 val

Using 2 dataloader workers

Logging results to /content/runs/roadeye_plate_single_class

Starting training for 80 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/80	3.88G	1.218	0.851	1.131	21	640: 100%
	Class	Images	Instances	Box(P	R	mAP50 mAP5
	all	1765	1840	0.947	0.892	0.941

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/80	5.13G	1.181	0.6445	1.117	26	640: 100%
	Class	Images	Instances	Box(P	R	mAP50 mAP5
	all	1765	1840	0.971	0.916	0.964

from google.colab import files

```
# This downloads best.pt directly to your computer
files.download('/content/runs/roadeye_plate_single_class/weights/best.pt')
```

```
!pip install ultralytics -q
import ultralytics
ultralytics.checks()
```

Ultralytics 8.4.69 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Setup complete ✅ (2 CPUs, 12.7 GB RAM, 47.4/112.6 GB disk)

```
pip install inference-sdk pillow pyyaml tqdm
```

```
Requirement already satisfied: inference-sdk in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (1
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (6
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.6
Requirement already satisfied: requests<3.0.0,>=2.33.0 in /usr/local/lib/python3.12
Requirement already satisfied: dataclasses-json~=0.6.0 in /usr/local/lib/python3.12
Requirement already satisfied: opencv-python<=4.10.0.84,>=4.8.1.78 in /usr/local/li
Requirement already satisfied: supervision>=0.26 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: numpy<2.4.0,>=2.0.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: aiohttp~=3.14.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: backoff~=2.2.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: py-cpuinfo~=9.0.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: typing_extensions>=4.4 in /usr/local/lib/python3.12/
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: marshmallow<4.0.0,>=3.18.0 in /usr/local/lib/python3
Requirement already satisfied: typing-inspect<1,>=0.4.0 in /usr/local/lib/python3.1
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.1
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: urllib3<3,>=1.26 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: certifi>=2023.5.7 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: defusedxml>=0.7.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: matplotlib>=3.6 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pydeprecate<0.10,>=0.9 in /usr/local/lib/python3.12/
Requirement already satisfied: scipy>=1.10 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: packaging>=17.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/di
Requirement already satisfied: mpyy-extensions>=0.3.0 in /usr/local/lib/python3.12/
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
```

```

from ultralytics import YOLO
!pip install roboflow
from roboflow import Roboflow
import yaml, os
from pathlib import Path

```

```

Requirement already satisfied: roboflow in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: idna==3.7 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cycler in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: opencv-python-headless==4.10.0.84 in /usr/local/lib/
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pi-heif<2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pillow-avif-plugin<2 in /usr/local/lib/python3.12/di
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: urllib3>=1.26.6 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: tqdm>=4.41.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: PyYAML>=5.3.1 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: requests_toolbelt in /usr/local/lib/python3.12/dist-
Requirement already satisfied: filetype in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typer<0.26,>=0.12.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: click>=8.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.1
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-package

```

```

from ultralytics import YOLO
from roboflow import Roboflow
import yaml, os
from pathlib import Path
MODEL_PATH = "best.pt"
model = YOLO(MODEL_PATH)
print("Loaded model architecture:")
print(f"  Task:          {model.task}")
print(f"  Classes:       {model.names}")
print(f"  Parameters: {sum(p.numel() for p in model.model.parameters()):,}")

```

```

Loaded model architecture:
  Task:          detect
  Classes:       {0: 'plate'}
  Parameters: 11,135,987

```

```

rf = Roboflow(api_key="bZewy443VlofM60vQSMi")

datasets = []
proj1 = rf.workspace("augmented-startups").project("vehicle-registration-plates-tr
ds1 = proj1.version(1).download("yolov8")
datasets.append(Path(ds1.location))

```

```

loading Roboflow workspace...
loading Roboflow project...

```

```

rf = Roboflow(api_key="bZewy443VlofM60vQSMi")
proj2 = rf.workspace("samrat-sahoo").project("license-plates-f8vsn")
version = proj2.version(5)
ds2 = version.download("yolov8")
datasets.append(Path(ds2.location))

```

```

loading Roboflow workspace...
loading Roboflow project...

```

```

from pathlib import Path
from PIL import Image
from inference_sdk import InferenceHTTPClient
import random
import shutil
import yaml
from tqdm import tqdm

API_KEY = "bZewy443VlofM60vQSMi"
WORKSPACE = "kirti-dnxr3"
WORKFLOW_ID = "find-vehicles-and-license-plates"

INPUT_DIR = Path("raw_images")
OUTPUT_DIR = Path("hardcoded_data")

CLASSES = ["vehicles", "license plates"]
CLASS_TO_ID = {name: i for i, name in enumerate(CLASSES)}

SPLITS = {
    "train": 0.8,
    "valid": 0.1,
    "test": 0.1,
}

CONFIDENCE = 0.5
IOU_THRESHOLD = 0.3

client = InferenceHTTPClient(
    api_url="https://serverless.roboflow.com",
    api_key=API_KEY,
)

def yolo_line(pred, img_w, img_h):

```

```

class_name = (
    pred.get("class")
    or pred.get("class_name")
    or pred.get("label")
)

if class_name not in CLASS_TO_ID:
    return None

x = float(pred["x"])
y = float(pred["y"])
w = float(pred["width"])
h = float(pred["height"])

x_center = x / img_w
y_center = y / img_h
width = w / img_w
height = h / img_h

# Clamp to valid YOLO range
x_center = min(max(x_center, 0), 1)
y_center = min(max(y_center, 0), 1)
width = min(max(width, 0), 1)
height = min(max(height, 0), 1)

return f"{CLASS_TO_ID[class_name]} {x_center:.6f} {y_center:.6f} {width:.6f} {"

def extract_predictions(workflow_result):
    """
    Handles common Roboflow Workflow output shapes.
    """
    if isinstance(workflow_result, list):
        workflow_result = workflow_result[0]

    preds = workflow_result.get("predictions", workflow_result)

    if isinstance(preds, dict) and "predictions" in preds:
        return preds["predictions"]

    if isinstance(preds, list):
        return preds

    return []

def assign_split(files):
    random.shuffle(files)

    n = len(files)
    train_end = int(n * SPLITS["train"])
    valid_end = train_end + int(n * SPLITS["valid"])

    return {
        "train": files[:train_end],
        "valid": files[train_end:valid_end]
    }

```



```

        "test": files[valid_end:],
    }
}

def main():
    image_files = [
        p for p in INPUT_DIR.glob("*")
        if p.suffix.lower() in [".jpg", ".jpeg", ".png", ".bmp", ".webp"]
    ]

    if not image_files:
        raise ValueError(f"No images found in {INPUT_DIR}")

    for split in SPLITS:
        (OUTPUT_DIR / split / "images").mkdir(parents=True, exist_ok=True)
        (OUTPUT_DIR / split / "labels").mkdir(parents=True, exist_ok=True)

    split_files = assign_split(image_files)

    for split, files in split_files.items():
        for img_path in tqdm(files, desc=f"Annotating {split}"):
            with Image.open(img_path) as im:
                img_w, img_h = im.size

                result = client.run_workflow(
                    workspace_name=WORKSPACE,
                    workflow_id=WORKFLOW_ID,
                    images={
                        "image": str(img_path)
                    },
                    parameters={
                        "confidence": CONFIDENCE,
                        "iou_threshold": IOU_THRESHOLD,
                        "class_agnostic_nms": False,
                        "max_detections": 1000,
                    },
                    use_cache=True,
                )

                predictions = extract_predictions(result)

                label_lines = []
                for pred in predictions:
                    line = yolo_line(pred, img_w, img_h)
                    if line is not None:
                        label_lines.append(line)

                out_img = OUTPUT_DIR / split / "images" / img_path.name
                out_label = OUTPUT_DIR / split / "labels" / f"{img_path.stem}.txt"

                shutil.copy2(img_path, out_img)
                out_label.write_text("\n".join(label_lines))

    data_yaml = {
        "path": str(OUTPUT_DIR.resolve())
    }

```

```

        path : str(OUTPUT_DIR.resolve()),
        "train": "train/images",
        "val": "valid/images",
        "test": "test/images",
        "names": {i: name for i, name in enumerate(CLASSES)},
    }

    with open(OUTPUT_DIR / "data.yaml", "w") as f:
        yaml.safe_dump(data_yaml, f, sort_keys=False)

    print(f"Done. YOLO dataset written to: {OUTPUT_DIR.resolve()}")

if __name__ == "__main__":
    main()

```

```

Annotating train: 100%|██████████| 25/25 [00:54<00:00, 2.17s/it]
Annotating valid: 100%|██████████| 3/3 [00:06<00:00, 2.00s/it]
Annotating test: 100%|██████████| 4/4 [00:06<00:00, 1.53s/it]Done. YOLO dataset wr

```

```
MY_HARD_DATA = Path("/content/hardcoded_data")
```

```

def force_single_class(root: Path):
    for split in ["train", "valid", "test"]:
        label_dir = root / split / "labels"
        if not label_dir.exists():
            continue
        for lf in label_dir.glob("*.txt"):
            lines = []
            for line in lf.read_text().strip().splitlines():
                parts = line.split()
                if len(parts) >= 5:
                    parts[0] = "0"
                    lines.append(" ".join(parts))
            lf.write_text("\n".join(lines) + "\n")
    print(f" Normalised: {root.name}")

for ds_path in datasets:
    force_single_class(ds_path)

```

```

Normalised: Vehicle-Registration-Plates-1
Normalised: License-Plates-5

```

```

COMBINED = Path("/content/combined_data")
for split in ["train", "valid"]:
    (COMBINED / split / "images").mkdir(parents=True, exist_ok=True)
    (COMBINED / split / "labels").mkdir(parents=True, exist_ok=True)

```

```
import shutil
```

```
def merge_split(split: str, sources: list[Path], extra: Path | None = None):
    all_sources = list(sources)
    if extra and (extra / split).exists():
        all_sources.append(extra)

    for src in all_sources:
        img_dir = src / split / "images"
        lbl_dir = src / split / "labels"
        if not img_dir.exists():
            continue
        for img in img_dir.glob("*."):
            lbl = lbl_dir / (img.stem + ".txt")
            dst_img = COMBINED / split / "images" / f"{src.name}_{img.name}"
            dst_lbl = COMBINED / split / "labels" / f"{src.name}_{img.stem}.txt"
            shutil.copy2(img, dst_img)
            if lbl.exists():
                shutil.copy2(lbl, dst_lbl)

merge_split("train", datasets, MY_HARD_DATA)
merge_split("valid", datasets, MY_HARD_DATA)

train_count = len(list((COMBINED / "train" / "images").glob("*.")))
valid_count = len(list((COMBINED / "valid" / "images").glob("*.")))
print(f"Combined dataset: {train_count} train | {valid_count} valid")
```

Combined dataset: 7180 train | 1838 valid

```
data_yaml = COMBINED / "data.yaml"
yaml.safe_dump({
    "path": str(COMBINED),
    "train": "train/images",
    "val": "valid/images",
    "nc": 1,
    "names": ["plate"],
}, open(data_yaml, "w"), sort_keys=False)
```

```
results = model.train(
    data      = str(data_yaml),
    epochs    = 40,
    imgsz     = 640,
    batch     = 16,
    patience  = 15,
    device    = 0,
    project   = "/content/runs",
    name      = "roadeye_finetune_v2",

    # — Learning rate (MOST IMPORTANT for fine-tuning) —————
    lr0       = 0.0005,          # start low – 10× lower than scratch (0.001)
    lrf       = 0.01,           # final lr = lr0 * lrf
    cos_lr    = True,
    warmup_epochs = 2,
```

```

# — Freeze backbone layers —————
# Keeps low-level feature detectors intact, only trains detection head
# Use 0 if you have 1000+ new images; use 15-20 for <300 new images
freeze      = 10,

# — Augmentation for real-world conditions —————
# Motion blur simulation (key for video frames)
# blur      = 0.3,          # random Gaussian blur
# Lighting variation (night / harsh sun)
hsv_v      = 0.6,          # brightness jitter ↑ from 0.5
hsv_s      = 0.5,
hsv_h      = 0.02,
# Geometric (angled dashcam shots)
degrees    = 5.0,          # rotation ↑ from 3.0
perspective = 0.001,       # slight perspective warp
shear      = 2.0,          # horizontal shear for angled plates
scale      = 0.5,          # ↑ from 0.3 – simulates distance variation
translate  = 0.1,
# Keep these off – plate text is not symmetric
fliplr     = 0.0,
flipud     = 0.0,
# Mosaic off at lower rate – we want single-plate focus
mosaic     = 0.3,
# Random erasing – simulates partial occlusion (mud, stickers)
erasing    = 0.3,

optimizer  = "AdamW",
plots      = True,
)

print("\nFine-tuning complete!")
print(f"Best mAP50:      {results.results_dict.get('metrics/mAP50(B)').:.4f}")
print(f"Best mAP50-95:   {results.results_dict.get('metrics/mAP50-95(B)').:.4f}")

```

Ultralytics 8.4.69 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augmen
 Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/'

	from	n	params	module
0	-1	1	928	ultralytics.nn.modules.conv.Conv
1	-1	1	18560	ultralytics.nn.modules.conv.Conv
2	-1	1	29056	ultralytics.nn.modules.block.C2f
3	-1	1	73984	ultralytics.nn.modules.conv.Conv
4	-1	2	197632	ultralytics.nn.modules.block.C2f
5	-1	1	295424	ultralytics.nn.modules.conv.Conv
6	-1	2	788480	ultralytics.nn.modules.block.C2f
7	-1	1	1180672	ultralytics.nn.modules.conv.Conv
8	-1	1	1838080	ultralytics.nn.modules.block.C2f
9	-1	1	656896	ultralytics.nn.modules.block.SPPF
10	-1	1	0	torch.nn.modules.upsampling.Upsample
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat
12	-1	1	591360	ultralytics.nn.modules.block.C2f
13	-1	1	0	torch.nn.modules.upsampling.Upsample
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat
15	-1	1	148224	ultralytics.nn.modules.block.C2f

```
--
16          -1  1    147712 ultralytics.nn.modules.conv.Conv
17      [-1, 12]  1         0 ultralytics.nn.modules.conv.Concat
18          -1  1    493056 ultralytics.nn.modules.block.C2f
19          -1  1    590336 ultralytics.nn.modules.conv.Conv
20      [-1,  9]  1         0 ultralytics.nn.modules.conv.Concat
21          -1  1    1969152 ultralytics.nn.modules.block.C2f
22      [15, 18, 21]  1    2116435 ultralytics.nn.modules.head.Detect
Model summary: 130 layers, 11,135,987 parameters, 11,135,971 gradients, 28.6 GFLOPs
```

Transferred 355/355 items from pretrained weights

```
Freezing layer 'model.0.conv.weight'
Freezing layer 'model.0.bn.weight'
Freezing layer 'model.0.bn.bias'
Freezing layer 'model.1.conv.weight'
Freezing layer 'model.1.bn.weight'
Freezing layer 'model.1.bn.bias'
Freezing layer 'model.2.cv1.conv.weight'
Freezing layer 'model.2.cv1.bn.weight'
Freezing layer 'model.2.cv1.bn.bias'
Freezing layer 'model.2.cv2.conv.weight'
Freezing layer 'model.2.cv2.bn.weight'
Freezing layer 'model.2.cv2.bn.bias'
Freezing layer 'model.2.m.0.cv1.conv.weight'
Freezing layer 'model.2.m.0.cv1.bn.weight'
Freezing layer 'model.2.m.0.cv1.bn.bias'
Freezing layer 'model.2.m.0.cv2.conv.weight'
Freezing layer 'model.2.m.0.cv2.bn.weight'
Freezing layer 'model.2.m.0.cv2.bn.bias'
Freezing layer 'model.3.conv.weight'
Freezing layer 'model.3.bn.weight'
Freezing layer 'model.3.bn.bias'
Freezing layer 'model.4.cv1.conv.weight'
Freezing layer 'model.4.cv1.bn.weight'
Freezing layer 'model.4.cv1.bn.bias'
Freezing layer 'model.4.cv2.conv.weight'
Freezing layer 'model.4.cv2.bn.weight'
Freezing layer 'model.4.cv2.bn.bias'
```

```
from google.colab import files
files.download("/content/runs/roadeye_finetune_v2/weights/best.pt")
```

```
from google.colab import files
files.download("/content/runs/roadeye_finetune_v2/args.yaml")
```

Start coding or [generate](#) with AI.

